

LEVEL 7

Chapter 8: Microservices

System Design Master Roadmap — 9 Years of Experience Edition

Compiled August 2026



Chapter 8: Microservices

← [Chapter 7: Messaging & Event-Driven](#) · [Index](#) · Next → [Chapter 9: API Design](#)

8.1 Service Boundaries — The Question That Actually Matters

Everything else in this chapter is plumbing. The genuinely hard, genuinely interview-worthy question is: **where do you draw the lines between services?** Get this wrong and every technique below (service discovery, tracing, deployment pipelines) just makes a badly-shaped system run smoothly, which doesn't fix the underlying problem.

The right heuristic: bounded contexts, not technical layers. Draw service boundaries around business capabilities that change together and are owned by one team (e.g., "Orders," "Payments," "Inventory," "Notifications") — not around technical layers (e.g., "the database service," "the validation service"), which forces every business change to touch multiple services and creates constant cross-service coupling. This is the core idea behind Domain-Driven Design's "bounded context," and naming it explicitly is a strong interview signal.

A good boundary test: if changing one business rule (e.g., "orders under \$10 don't get free shipping") requires deploying two or more services in careful coordination, the boundary is probably wrong. Well-drawn boundaries mean most changes are containable within one service.

Database-per-service

Each microservice owns its own database, and — this is the strict part — **no other service is allowed to touch it directly**, not even for a read. All access goes through the owning service's API. This is what actually makes services independently deployable: a schema change is a local decision, not a cross-team negotiation.

Trade-off this creates: you lose easy cross-entity joins and multi-entity ACID transactions across service boundaries — which is exactly why Chapter 6's Saga pattern exists. This trade-off is the single biggest reason "just use microservices" isn't free — you're deliberately giving up relational-database conveniences you had in a monolith, in exchange for independent scaling and deployment.

8.2 Synchronous vs. Asynchronous Communication

	Synchronous (REST/gRPC request-response)	Asynchronous (events via Kafka/SQS)
Caller behavior	Waits for a response	Fires an event and moves on
Coupling	Tighter — caller needs the callee to be up and responsive <i>right now</i>	Looser — callee processes when ready; caller doesn't need callee to be up at all
Failure mode	Caller directly feels callee's slowness/downtime (needs timeouts, circuit breakers, retries from Chapter 6)	Callee's downtime just means a processing delay — the event sits in the queue
Use when	You need an immediate answer to proceed (e.g., "is this payment authorized" before showing a confirmation page)	The action doesn't need to block the user's request (e.g., "send a confirmation email" shouldn't make checkout wait)

Interview question: "A checkout flow needs to: validate payment, reserve inventory, send a confirmation email, and update analytics. Which of these are sync, which are async?"

Ideal senior answer: "Payment validation and inventory reservation are synchronous — the user genuinely needs to know 'did this succeed' before the checkout flow can complete, and both can legitimately fail in ways that change what the user sees next. Confirmation email and analytics are asynchronous — the user shouldn't wait an extra 300ms for an email provider's API just to see an order confirmation, and if the email service is briefly down, that shouldn't fail the checkout at all. I'd publish an 'order confirmed' event after the synchronous steps succeed, and let email/analytics consume it independently."

8.3 Service Discovery

The problem: in a system where services scale up/down and get rescheduled constantly (especially on Kubernetes), a service's IP address is not stable — hardcoding `10.0.4.12:8080` breaks the moment that pod is rescheduled. Service discovery is how a caller finds a *current, healthy* instance of the service it needs to call, by name.

- **Client-side discovery:** the calling service queries a registry (e.g., Consul, Eureka) directly and picks an instance itself, often with client-side load balancing.
- **Server-side discovery:** the calling service just calls a stable name/endpoint (a Kubernetes Service, or a load balancer), and something else (kube-proxy, a service mesh sidecar) resolves that to a real instance behind the scenes. **This is what you already use on EKS** — a Kubernetes `Service` object is server-side discovery, backed

by `kube-dns` /CoreDNS resolving a stable DNS name to the current healthy pod IPs via `Endpoints`.

8.4 Configuration, Secrets, and Distributed Tracing

Configuration: externalize environment-specific config (feature flags, connection strings, timeouts) from code, so the same build artifact can be promoted through dev → staging → prod without rebuilding — typically via environment variables, a config service, or (on Kubernetes) ConfigMaps.

Secrets: credentials, API keys, certificates — never in code or plain config files. Use a dedicated secrets manager (AWS Secrets Manager, HashiCorp Vault, Kubernetes Secrets backed by encryption at rest) with tightly scoped access via IAM roles, and rotate regularly. This is worth stating explicitly in any interview touching security or fintech.

Distributed tracing: in a request that fans out across 8 microservices, "why was this request slow" is unanswerable from any single service's logs alone. Distributed tracing (OpenTelemetry — which you already use — feeding Jaeger/Tempo/X-Ray) attaches a **trace ID** to the original request and propagates it through every downstream call, with each service recording a **span** (its portion of the work, with timing). The result is a single flame-graph-style view of the entire request's path across every service, showing exactly where time was spent. Pair this with a **correlation ID** in logs (often the same trace ID) so you can jump from "this trace was slow" to "here are the exact log lines from every service involved."

Health checks: every service should expose a liveness check ("is this process alive at all — restart me if not") and a readiness check ("am I currently able to serve traffic — don't route to me if not, but don't restart me either, e.g., I'm still warming a cache"). Kubernetes' `livenessProbe` and `readinessProbe` map directly to this distinction, which you've likely configured yourself.

8.5 Deployment, Versioning, Backward Compatibility

Independent deployability is the actual point of microservices — if deploying service A still requires coordinating a simultaneous deploy of service B, you haven't achieved the main benefit, regardless of how many separate repos/containers you have.

This makes backward compatibility a hard requirement, not a nice-to-have: service A might be calling an old version of service B's API for minutes or hours during a rolling deployment (especially with canary/blue-green strategies) — so a new version of an API must not break existing callers. Practically: add new fields as optional, never remove or repurpose an existing field's meaning, version breaking changes explicitly (`/v2/orders`)

rather than mutating `/v1/orders` in place, and for events, treat your message schemas with the same discipline (consider a schema registry, like Confluent Schema Registry, if using Kafka with Avro/Protobuf).

8.6 Failure Modes Unique to Microservices

Cascading failures: service A calls B calls C. C slows down. B's threads pile up waiting on C, exhausting B's own capacity, so B becomes slow/unresponsive to A. A's threads pile up waiting on B. The failure "cascades" backward through the call chain, and a single slow leaf service can take down the entire system if nothing interrupts the chain. **This is exactly what circuit breakers, bulkheads, and timeouts (Chapter 6) exist to stop** — this chapter's content and Chapter 6's content are two sides of the same coin, and interviewers will connect them.

Distributed transaction problems: covered fully in Chapter 6 (Saga/2PC) — the short version is that "update three services' databases atomically" isn't naturally possible once each service owns its own database, and pretending otherwise (or reaching for 2PC) creates more problems than it solves.

Service dependency problems (the "distributed monolith" trap): if service A can't function at all without a synchronous call to B, and B can't function without C, you've built a system that's *organizationally* distributed (separate teams, separate repos, separate deploys) but *operationally* coupled exactly like a monolith — you've paid all of microservices' complexity cost and gotten none of the independent-availability benefit. The fix is architectural: minimize synchronous dependency chains, prefer async event-driven communication where the caller doesn't need an immediate answer, and design each service to degrade gracefully (return cached/default data) rather than fail hard when a non-critical dependency is unavailable.

Interview question: "How do you prevent one struggling microservice from taking down the whole system?"

Ideal senior answer: "Layered defense, not one fix: timeouts on every network call so nothing waits forever; circuit breakers so repeated failures stop generating more load on a struggling dependency; bulkheads so a slow dependency's calls don't exhaust a thread pool shared with unrelated calls; and, architecturally, minimizing how many services are on the synchronous critical path for any single user action in the first place — every synchronous hop is a chance for someone else's bad day to become yours. Where a dependency is genuinely non-critical to the immediate user-facing outcome, I'd make that call asynchronous or have a documented graceful-degradation fallback rather than fail the whole request."

Chapter 8 Interview Drill

1. Explain "bounded context" as a service-boundary heuristic, with a concrete counter-example (a bad, technically-layered boundary).
2. What specific database convenience do you give up with database-per-service, and what pattern compensates for it?
3. Walk through client-side vs. server-side service discovery, and which one Kubernetes gives you by default.
4. Explain a cascading failure scenario across three services, and name the three specific mitigations.
5. What is a "distributed monolith," and how would you recognize you've accidentally built one?

Next → [Chapter 9: API Design](#) — REST, GraphQL, gRPC, pagination, idempotency keys, and good vs. bad API examples.